

Machine Learning - Project

Tin Salopek

May 10, 2026

1 Literature study - Task 1

The biggest influences on this work were [YS09] and [Chr24]. In these books there are good explanations of basic game theoretic concepts and their connection to reinforcement learning. Then in [ea15] there are explanations of replicator dynamics and on simple examples such as Cross learning it is explained how to derive the dynamics equations. And finally, lenient algorithms are based on [LP07].

2 Pareto optimal profiles and Nash equilibria of matrix games

For each of the 4 games a matrix is given which describes players, their possible actions and utility functions. This is equivalent to a normal-form game tuple (N, A, u) as in [YS09]. N is the set of integers where each element represents a player. A is the set which represents the cartesian product of each player's possible actions and u is the utility function which takes an input an action profile (element of A) and returns a real number.

2.1 Important definitions

Definition 2.1. A strategy s of a player i is the probability distribution over its actions and S_i is the set of all strategies:

$$s \in S_i$$

Definition 2.2. Set of all strategy profiles S is a cartesian product of sets of strategies of every player:

$$S = S_1 \times \dots \times S_N$$

Definition 2.3. Expected utility for a player i given a strategy profile s :

$$e_i(s) = \sum_{a \in A} u_i(a) \prod_{j=1}^n s_j(a_j), \quad s \in S$$

Definition 2.4. Strategy profile s dominates strategy profile s' if:

a)

$$\forall i \in N : e_i(s) \geq e_i(s')$$

b)

$$\exists j \in N : e_j(s) > e_j(s')$$

If a strategy profile is not Pareto dominated then it is Pareto optimal.

Definition 2.5. Best response of a player i to the strategy profile s_{i-} is the strategy s_* such that

$$e_i(s_*, s_{i-}) \geq e_i(s_i, s_{i-}) \quad \forall s_i$$

If all players are playing the best response strategies then the game is in a Nash equilibrium.

Definitions are taken from [YS09] and [Chr24] with a slight change in notation.

2.2 Game 1 - Stag hunt

Stag hunt in a normal form game notation is:

$$N = \{1, 2\}, \quad A = \{S, H\} \times \{S, H\}, \quad u(S, S) = (1, 1), \quad u(S, H) = (0, \frac{2}{3}), \quad u(H, S) = (\frac{2}{3}, 0), \quad u(H, H) = (\frac{2}{3}, \frac{2}{3})$$

Let s_1 be player 1 mixed strategy

$$s_1 \sim \begin{pmatrix} S & H \\ p & 1-p \end{pmatrix}$$

and s_2 player 2 mixed strategy

$$s_2 \sim \begin{pmatrix} S & H \\ q & 1-q \end{pmatrix}$$

Then it is possible to write the expected return of player 1 and 2 given the strategy profile $s = (s_1, s_2)$ by definition 2.3 :

$$e_1(s) = \sum_{a \in \{(S,S), (S,H), (H,S), (H,H)\}} u_1(a) \prod_{j=1}^n s_j(a_j) = 1(pq) + 0(p(1-q)) + \frac{2}{3}((1-p)q) + \frac{2}{3}((1-p)(1-q)) = pq - \frac{2}{3}p + \frac{2}{3}$$

Similarly for e_2 :

$$e_2(s) = pq - \frac{2}{3}q + \frac{2}{3}$$

In order to find a Pareto optimal strategy profile we can solve two constrained optimization problems where we maximize the functions $e_1(s)$ and $e_2(s)$ with constraints $(p, q) \in [0, 1] \times [0, 1]$. A solution to both of these problems is $\begin{pmatrix} p \\ q \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$. This solution is one Pareto optimal mixed strategy profile.

If we restrict only to pure strategies it is easily shown that it dominates all 3 other pure profiles so then it is the only one. To find Nash equilibria we can observe the same functions e_1 and e_2 . Now the two functions we are trying to optimize are e_1 on $p \in [0, 1]$ and e_2 on $q \in [0, 1]$. The solutions we get for Nash equilibria are strategy profiles: $(s_1 \sim \begin{pmatrix} S & H \\ 1 & 0 \end{pmatrix}, s_2 \sim \begin{pmatrix} S & H \\ 1 & 0 \end{pmatrix}), (s_1 \sim \begin{pmatrix} S & H \\ 0 & 1 \end{pmatrix}, s_2 \sim \begin{pmatrix} S & H \\ 0 & 1 \end{pmatrix}), (s_1 \sim \begin{pmatrix} S & H \\ \frac{2}{3} & \frac{1}{3} \end{pmatrix}, s_2 \sim \begin{pmatrix} S & H \\ \frac{2}{3} & \frac{1}{3} \end{pmatrix})$. For the other 3 games the reasoning is similar.

2.3 Game 2 - Subsidy game

Here in terms of Pareto optimality the situation is similar to the Stag Hunt. One Pareto optimal strategy profile is $((s_1 \sim \begin{pmatrix} S_1 & S_2 \\ 1 & 0 \end{pmatrix}, s_2 \sim \begin{pmatrix} S_1 & S_2 \\ 1 & 0 \end{pmatrix}))$. This is also the only pure strategy Pareto

optimal profile. Nash equilibria for this game are profiles: $(s_1 \sim \begin{pmatrix} S_1 & S_2 \\ 1 & 0 \end{pmatrix}, s_2 \sim \begin{pmatrix} S_1 & S_2 \\ 1 & 0 \end{pmatrix}), (s_1 \sim \begin{pmatrix} S_1 & S_2 \\ 0 & 1 \end{pmatrix}, s_2 \sim \begin{pmatrix} S_1 & S_2 \\ 0 & 1 \end{pmatrix}), (s_1 \sim \begin{pmatrix} S_1 & S_2 \\ \frac{10}{11} & \frac{1}{11} \end{pmatrix}, s_2 \sim \begin{pmatrix} S_1 & S_2 \\ \frac{10}{11} & \frac{1}{11} \end{pmatrix})$.

2.4 Game 3 - Prisoner's dilemma

Pareto optimal strategy profiles are $(s_1 \sim \begin{pmatrix} C & D \\ 1 & 0 \end{pmatrix}, s_2 \sim \begin{pmatrix} C & D \\ 0 & 1 \end{pmatrix}), (s_1 \sim \begin{pmatrix} C & D \\ 0 & 1 \end{pmatrix}, s_2 \sim \begin{pmatrix} C & D \\ 1 & 0 \end{pmatrix}), (s_1 \sim \begin{pmatrix} C & D \\ 1 & 0 \end{pmatrix}, s_2 \sim \begin{pmatrix} C & D \\ 1 & 0 \end{pmatrix})$. Nash equilibrium is the profile: $(s_1 \sim \begin{pmatrix} C & D \\ 0 & 1 \end{pmatrix}, s_2 \sim \begin{pmatrix} C & D \\ 0 & 1 \end{pmatrix})$

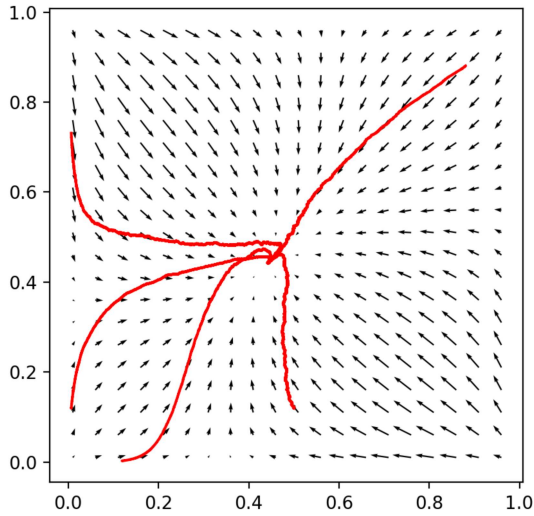
2.5 Game 4 - Biased Rock-Paper-Scissors

Here 2 situations are considered, namely, one-population dynamics and two-population dynamics. If we consider two-population dynamics than the Pareto optimal strategy profile is $(s_1 \sim \begin{pmatrix} R & P & S \\ 0 & 0 & 1 \end{pmatrix}, s_2 \sim \begin{pmatrix} R & P & S \\ 0 & 1 & 0 \end{pmatrix})$. This is also the Nash equilibrium. For one population dynamics of this game all profiles are Pareto optimal and Nash equilibria by definitions 2.4 and 2.5.

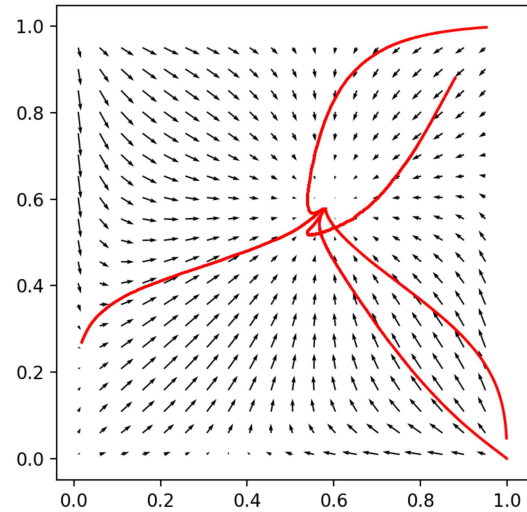
3 Learning trajectories and replicator dynamics - Task 2

Here the learning trajectories and replicator dynamics are plotted for all 4 games when applying Boltzmann Q-learning and lenient Boltzmann Q-learning. Replicator dynamics for those two algorithms are derived in [LP07]. For epsilon greedy Q-learning there are no plots because the exploration strategy can only be $s \sim \begin{pmatrix} action1 & action2 \\ 1 - \epsilon + \epsilon/2 & \epsilon/2 \end{pmatrix}$ or $s \sim \begin{pmatrix} action1 & action2 \\ \epsilon/2 & 1 - \epsilon + \epsilon/2 \end{pmatrix}$ which means the strategy simplex is reduced to those 2 points. Therefore for the epsilon greedy Q-learning only the convergent strategy is written.

3.1 Game 1 - Stag hunt



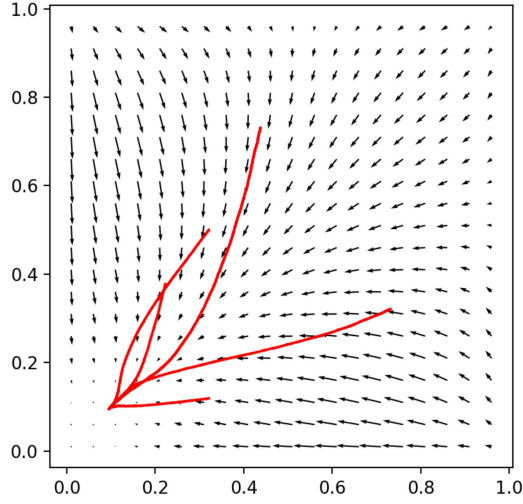
(a) Boltzmann Q-learning, $T = 1$, optimistic starting Q-values



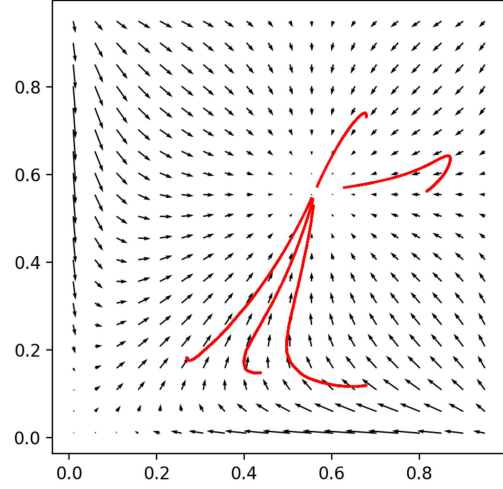
(b) Lenient Boltzmann Q-learning, $T = 1$, $N = 10$, optimistic starting Q-values

Epsilon greedy Q-learning with epsilon of 0.1 sometimes converges to $((s_1 \sim \begin{pmatrix} S & H \\ 0.05 & 0.95 \end{pmatrix}, s_2 \sim \begin{pmatrix} S & H \\ 0.05 & 0.095 \end{pmatrix}))$ and sometimes to $((s_1 \sim \begin{pmatrix} S & H \\ 0.95 & 0.05 \end{pmatrix}, s_2 \sim \begin{pmatrix} S & H \\ 0.95 & 0.05 \end{pmatrix}))$. It converges to the first point more often.

3.2 Game 2 - Subsidy game



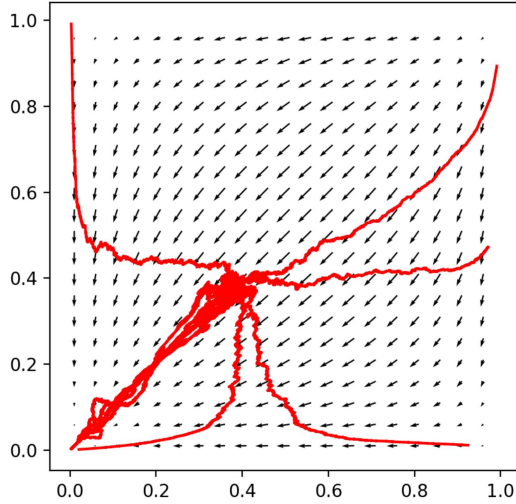
(a) Boltzmann Q-learning, $T = 4$, neutral starting Q-values



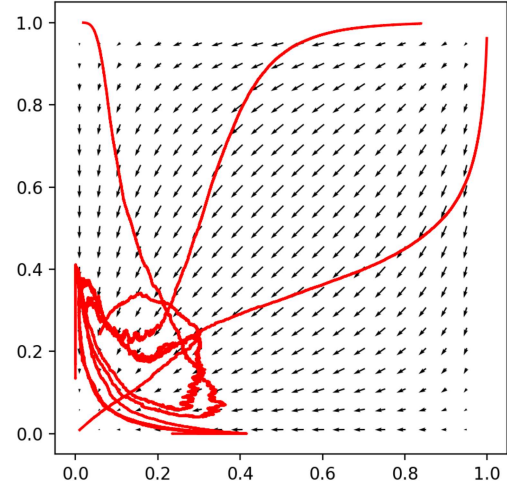
(b) Lenient Boltzmann Q-learning, $T = 4$, $N = 10$, neutral starting Q-values

Epsilon greedy Q-learning with epsilon of 0.1 sometimes converges to $((s_1 \sim \begin{pmatrix} S_1 & S_2 \\ 0.05 & 0.95 \end{pmatrix}, s_2 \sim \begin{pmatrix} S_1 & S_2 \\ 0.05 & 0.95 \end{pmatrix}))$ and sometimes to $((s_1 \sim \begin{pmatrix} S_1 & S_2 \\ 0.95 & 0.05 \end{pmatrix}, s_2 \sim \begin{pmatrix} S_1 & S_2 \\ 0.95 & 0.05 \end{pmatrix}))$. It converges to the first point more often.

3.3 Game 3 - Prisoner's dilemma



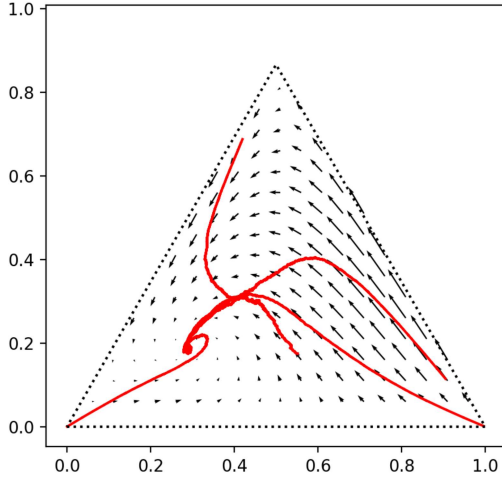
(a) Boltzmann Q-learning, $T = 0.1$, optimistic starting Q-values



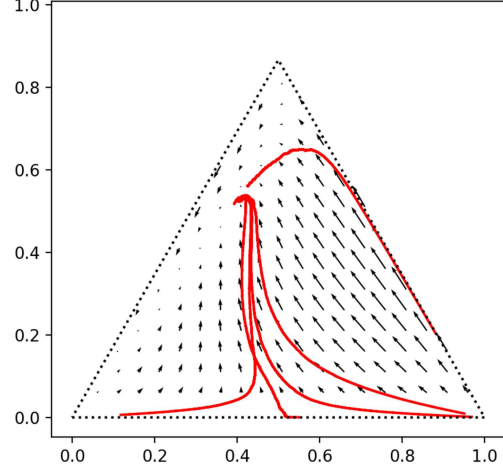
(b) Lenient Boltzmann Q-learning, $T = 0.1$, $N = 10$, optimistic starting Q-values

Epsilon greedy Q-learning with epsilon of 0.1 always converges to $((s_1 \sim \begin{pmatrix} C & D \\ 0.05 & 0.95 \end{pmatrix}, s_2 \sim \begin{pmatrix} C & D \\ 0.05 & 0.95 \end{pmatrix}))$

3.4 Game 4 - Biased Rock Paper Scissors



(a) Boltzmann Q-learning, $T = 0.1$, neutral starting Q-values



(b) Lenient Boltzmann Q-learning, $T = 0.1$, $N = 10$, neutral starting Q-values

Epsilon greedy Q-learning with epsilon of 0.1 always converges to equal Q-values for rock and paper and a smaller Q-value for scissors.

3.5 Analysis of learning trajectories and convergence

The goal of this section is to see how different reinforcement learning algorithms influence the points of convergence for four benchmark games. Big focus is also on how to improve coordination when that is possible. It is important to note: in this analysis, when we say that the algorithm converged to a pure strategy profile it does not mean that exploration strategies converge to those pure strategies. Once the learning is done we can take the largest Q value, assign probability of 1 to it's action and that is the final learned strategy.

We can see that the games Stag Hunt and Subsidy behave similarly: when Q-learning with Boltzmann exploration strategy is applied they converge to a point which is a Nash equilibrium but it is not a Pareto optimal profile. When epsilon greedy is applied then they mostly converge to a worse Nash equilibrium and rarely to a better one. When leniency is used then they converge to a Pareto optimal state and a state which is best in terms of coordination.

For Prisoner's dilemma, all 3 algorithms converge to the the only existent Nash equilibrium. However that is not a Pareto optimal state and it is not a good state in terms of coordination.

For Rock Paper Scissors the point of convergence changes when leniency is introduced but because the players use the same policy it can never converge to the best point for coordination.

4 Knights Archers Zombies - Task 3

4.1 Computer vision pipeline

4.1.1 Zombie detection

In order to determine the positions of zombies in a pixel based observation of size 1280 x 720 a pre-trained Yolo-v4 model is used. [AB20] The model is then fine-tuned on a dataset of generated frames from the game. When training pretrained models there is not a lot of design choices which can be made. The architecture alone is modified only in regards to the number of classes that need to be detected which is in this case 1. However it is important to generate a good dataset and here a dataset

of 3000 images and labels was generated, and in those there was 500 datapoints for each distortion level. For the preprocessing, observation images need to be resized to size 416 x 416 to be valid input to a model. For the post processing only the bounding boxes with confidence higher than some threshold are taken and rest are discarded.

In table 2 recall is showed for all 6 distortion levels. It is calculated by first calculating the intersection over union IoU for all the predicted boxes. Then it is counted the number of those boxes that have IoU larger than 0.5. Then that count is divided by the real number of zombies. This way of evaluating does not take into account the precision of the model, so a simple way to increase the score for level 5 would be to lower the confidence threshold to increase the number of true positives.

distortion level	recall
0	0.9792
1	0.9641
2	0.9526
3	0.9689
4	0.9607
5	0.3137

Table 1: Recall of zombie detection on different distortion levels, evaluated for 50 images for each level, with confidence threshold 0.8

4.1.2 Angle estimation

To estimate an angle of the player a convolutional neural network is used. The architecture is shown in figure 5. Activations function used in each layer is ReLu except for the last one where tanh is used. Negative cosine similarity is used as a loss function. Training dataset is generated by observing frames from the game, cropping the image on a 41 x 41 area around the agent 6 and grabbing the heading in x and y directions as a label.

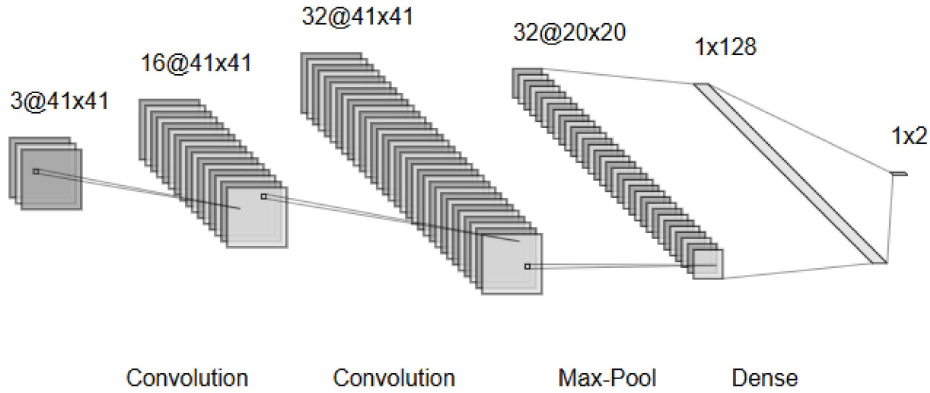


Figure 5: Angle estimation model architecture

Since the game always starts at the same state and action transitions for the agents are deterministic it is possible to just track the position and the angle of both agents. In that case we just get completely accurate agents positions and angles and we do not need to estimate angles.

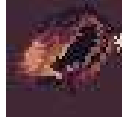


Figure 6: Example 41x41 input to the angle model

distortion level	average angle error in degrees
0	2.16
1	1.78
2	1.61
3	1.47
4	1.92
5	12.81

Table 2: Accuracy of angle estimation on different distortion levels, evaluated for 1000 images for each level

4.2 Proximal Policy Optimization

4.2.1 Feature enigneering

Once zombie positions and angle of the agent are extracted from pixel observations it is possible to construct features that encode state of the game. The feature vector is

$$\mathbf{x} = (x, y, dx_t, dy_t, h_x, h_y, dx_1, dy_1, f_1, \dots, dx_5, dy_5, f_5)$$

- x — x-position of the agent
- y — y-position of the agent
- dx_t — relative x position of the teammate
- dy_t — relative y position of the teammate
- h_x — x component of the heading
- h_y — y component of the heading
- dx_i — relative x position of zombie i
- dy_i — relative y position of zombie i
- f_i — flag which says if a zombie is in this spot

The flags which determine the presence of zombies can be 0 or 1 and all the other values are normaalized to a range from -1 to 1. The zombies are also sorted by their y value so that the first zombie slot laways represents the zombie with largest urgency.

4.2.2 Reward shaping

One of the trained agent pairs is trained with a prosocial reward scheme.[AP17] in order to try to induce coordination. The reward is changed so that agent a_i gets reward $R(a_i) = 0.6R(a_i) + 0.4R(a_j)$.

4.2.3 Algorithm hyperparameters

PPO hyperparameters used in training are :

- batch size = 4096
- learning rate = 10^{-4}
- $\gamma = 0.99$

- gradient clip = 0.5
- entropy coefficient = 0.005
- value function clip = 10
- $\epsilon = 0.2$
- $\lambda = 0.95$

4.3 Comparison with the baselines

In this section, agents are compared to two baselines: random agent and diagonal agent. Random agent uniformly samples actions from action space. Diagonal agent moves to its assigned corner, turns towards its diagonal and then chooses shoot always 8. The agents trained with a default reward scheme(the one that hits the zombie gets 1 point, other 0) learn to always go for the zombie that is furthest down. On the other hand, the agents trained with prosocial reward scheme learn to sometimes let the teammate go for the lowest zombie and go for another zombie that is on their side of the map. This is interesting because the agent doesn't know the heading of it's teammate so he has to "trust" it in some way and we also didn't specify beforehand which zombie a specific agent should aim. In 7 it is possible to get rewards larger than 120 because the maximum episode length is doubled (it was doubled also for training). The reason the tournament results are not 120 is because there is high variability in results, sometimes they reach higher than 120 and sometimes lower.

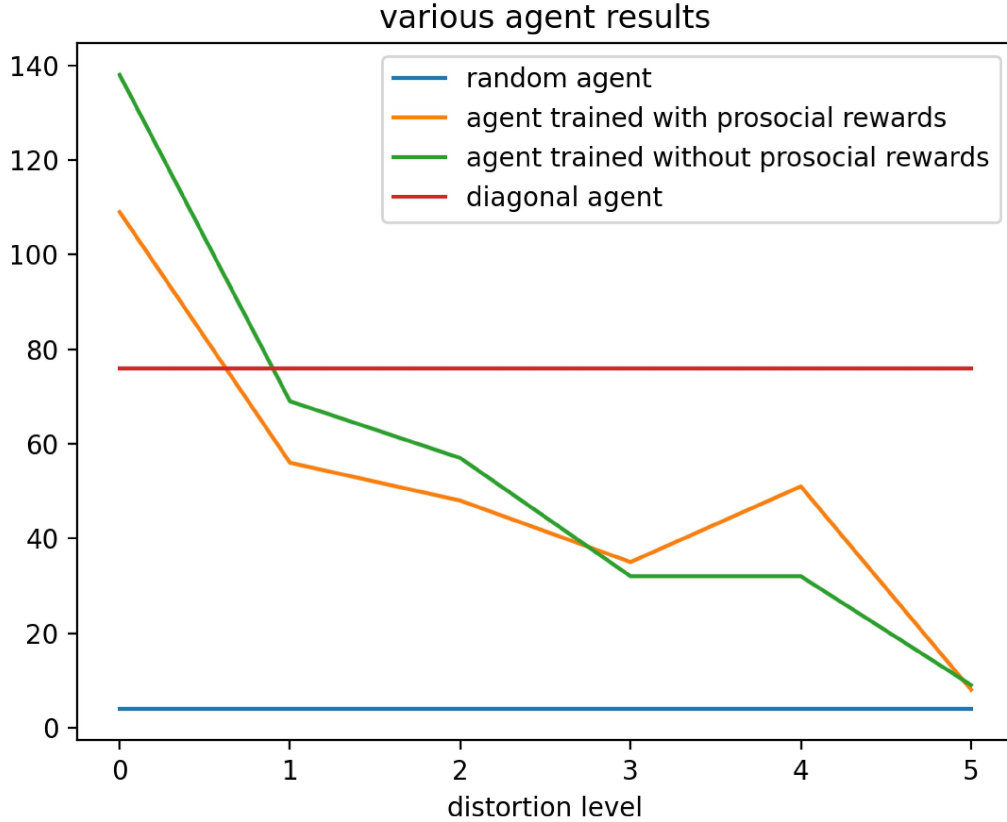


Figure 7: Results

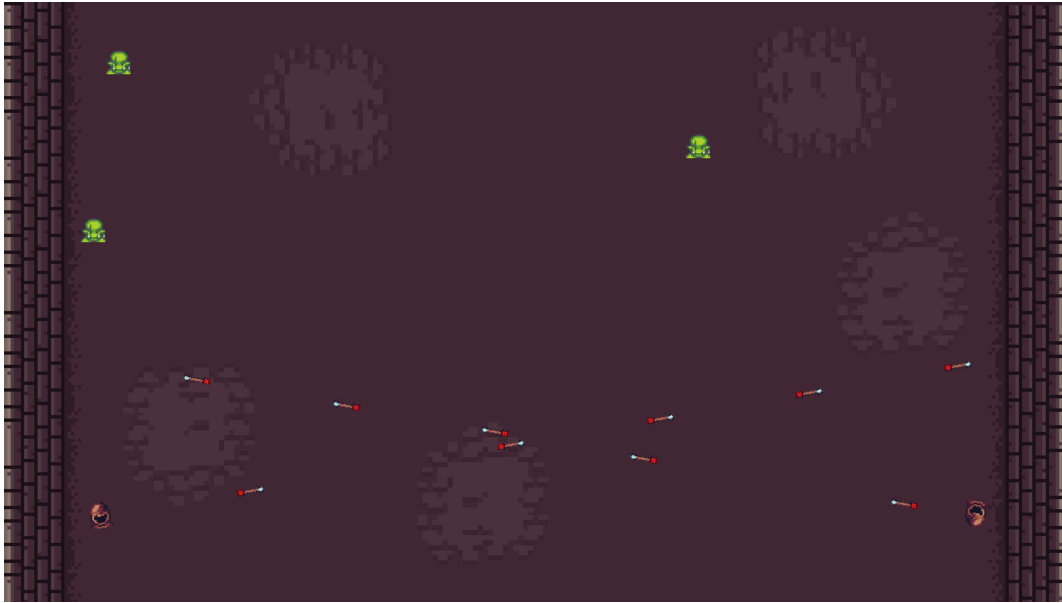


Figure 8: Diagonal agent

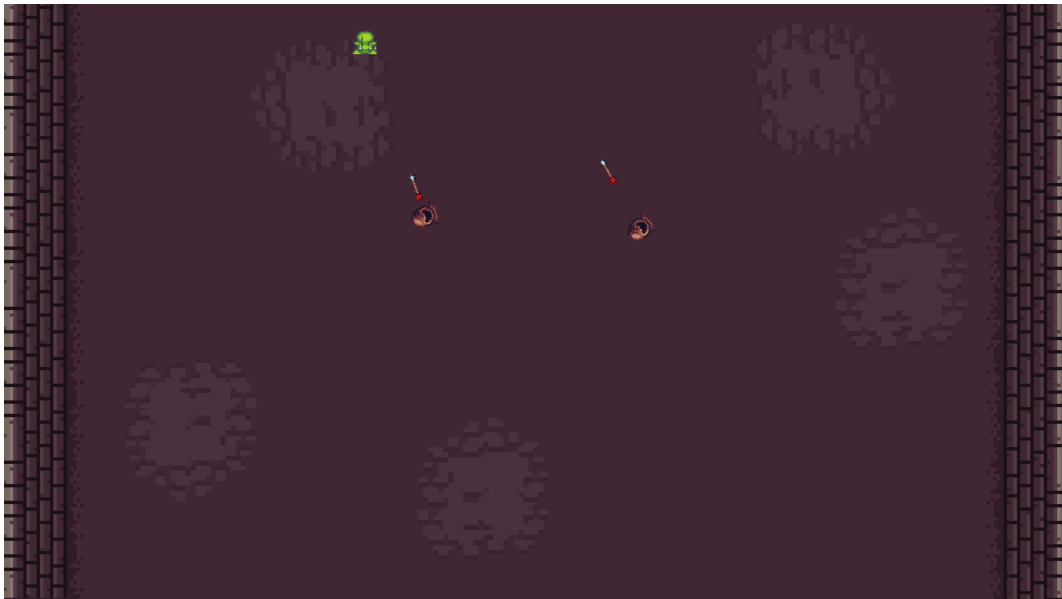


Figure 9: Positioning of agents trained with PPO

5 Additional info

For the visualization of the angle estimation architecture <https://alexlenail.me/NN-SVG/index.html> is used.

Code for the agents is located in: `/cw/lvs/NoCsBack/vakken/H0T25A/ml-project/students/r1085613/ml-project-2025-2026-main/`

Code for the matrix games is in: `/cw/lvs/NoCsBack/vakken/H0T25A/ml-project/students/r1085613/ml-project-2025-2026-main/MatrixGames`

References

- [AB20] Hong-Yuan Mark Liao Alexey Bochkovskiy, Chien-Yao Wang. Yolov4: Optimal speed and accuracy of object detection. 2020.
- [AP17] Adam Lerer Alexander Peysakhovich. Prosocial learning agents solve generalized stag hunts better than selfish ones. 2017.
- [Chr24] Lukas Schäfer Stefano V. Albrecht Filippas Christianos. Multi-agent reinforcement learning: Foundations and modern approaches. 2024.
- [ea15] Dan Bloembergen et al. Evolutionary dynamics of multi-agent learning: A survey”. in: J. artif. intell. res. (jair) 53 (2015), pp. 659–697. 2015.
- [LP07] Karl Tuyls Liviu Panait. Theoretical advantages of lenient q-learners: An evolutionary game theoretic perspective. 2007.
- [YS09] Kevin Leyton-Brown Yoav Shoham. Multiagent systems: Algorithmic, game-theoretic, and logical foundations. 2009.